🚀 The Complete Array Interview Handbook By Srinidhi Chintala @techstoriesofsrinidhi – Software Engineer at Microsoft

From Zero to FAANG

Pattern Recognition • Interview Triggers • Practice Roadmap

> 📘 *"Arrays are not just a topic. Arrays are the language interviewers use to test how you think."*

📖 Table of Contents

PART I — THE FOUNDATION

Chapter 1: What Interviewers Actually Test

Most beginners think that Data Structures & Algorithms interviews are about **memorizing solutions**. This is the single biggest misconception that keeps people stuck at the "beginner" stage forever.

Let's break the illusion right now.

> 🎯 **Interview Insight** Interviewers do **not** care whether you have seen a problem before. They care whether you can **think** in front of them. A memorized answer with no reasoning is actually a *red flag*, not a green one.

So what exactly is being tested when you're asked "Find the maximum subarray sum" or "Find the two numbers that add up to a target"? Let's go one layer deeper.

1.1 Pattern Recognition

Every array problem — no matter how it is worded — is a **disguised version of a known pattern**. FAANG interviewers don't invent brand-new algorithms during a 45-minute interview. Instead, they pick a well-known pattern and **change the story** around it.

For example:

Question as Asked	Real Pattern Hiding Inside
"Find the longest substring without repeating characters"	Sliding Window
"Find if two numbers in a sorted array add up to a target"	Two Pointer
"Find the sum of elements between index i and j, multiple times"	Prefix Sum
"Find the maximum sum of a contiguous subarray"	Kadane's Algorithm
"Find the number that appears more than n/2 times"	Boyer-Moore Voting
"You are given numbers from 1 to N with one missing, find it"	Cyclic Sort

This is why **pattern recognition is 70% of your interview score**. If you can label the problem correctly within the first 2 minutes, you have already won half the battle — coding it becomes a formality.

> 💡 **Remember** The goal of practicing 100+ problems is NOT to memorize 100+ solutions. The goal is to train your brain to recognize **10–12 patterns** inside hundreds of different problem descriptions.

1.2 Optimization Thinking

The second thing interviewers test is whether you **naturally move from a slow solution to a fast one**, without being told to.

Almost every array problem has multiple "levels" of solving it:

Level	Typical Complexity	What it Shows
Level 0	Doesn't work / wrong logic	Needs improvement
Level 1 (Brute Force)	$O(n^2)$ or $O(n^3)$	Basic understanding
Level 2 (Better)	O(n log n)	Awareness of sorting/searching tricks
Level 3 (Optimal)	O(n) or O(1) extra space	Strong DSA fundamentals

A candidate who jumps **straight** to the O(n) solution without explaining the journey often seems like they memorized it. A candidate who says:

> "The brute force way is to check every pair, which is $O(n^2)$. But since we only need one pass, I can use a HashMap to store what I've already seen, bringing it down to O(n)."

…sounds like an **engineer**, not a memorizer. This is exactly what interviewers are listening for.

> ✅ **Interview Insight** Always say your brute force approach OUT LOUD first — even if you know the optimal one. It shows your thought process and buys you time to plan the optimal code.

1.3 Brute Force → Optimal (The Golden Habit)

This is the single most important **habit** to build before your interview. Let's understand it with a simple example.

Problem: Find two numbers in an array that add up to a target.

Brute Force (Level 1): Check every pair using two nested loops.

```
for i in range(n):
    for j in range(i+1, n):
        if arr[i] + arr[j] == target:
            return [i, j]
```

Time Complexity: **$O(n^2)$** — Space: **O(1)**

Optimal (Level 3): Use a HashMap to remember numbers you've already seen.

```
seen = {}
for i, num in enumerate(arr):
    need = target - num
    if need in seen:
        return [seen[need], i]
    seen[num] = i
```

Time Complexity: **O(n)** — Space: **O(n)**

Notice the transformation: we traded a small amount of **extra space** to save a huge amount of **time**. This trade-off — **time vs. space** — is the heart of almost every optimization in array problems.

> ⚠️ **Common Mistake** Beginners jump to writing code before explaining their approach. Always explain your brute force idea, then explain your optimization idea, and only THEN start typing.

1.4 Why Arrays Are the Most Important Topic

You might wonder — why does an entire handbook exist just for arrays? Because:

1. **Arrays are the foundation of every other data structure.** Strings, Stacks, Queues, HashMaps, Heaps — all are built using arrays internally.
2. **Almost 40–50% of interview questions at FAANG+ companies are array-based** or use array-thinking (sliding window, two pointer, prefix sum) inside strings, matrices, and linked lists too.
3. **Arrays test raw problem-solving ability** without needing fancy data structure knowledge — which is exactly why they are asked in the very first round.
4. **Every advanced topic (DP, Graphs, Trees) reuses array patterns.** Kadane's algorithm is literally a 1D Dynamic Programming problem. Sliding Window appears in Graph BFS-style problems. Two Pointer appears in Linked Lists.

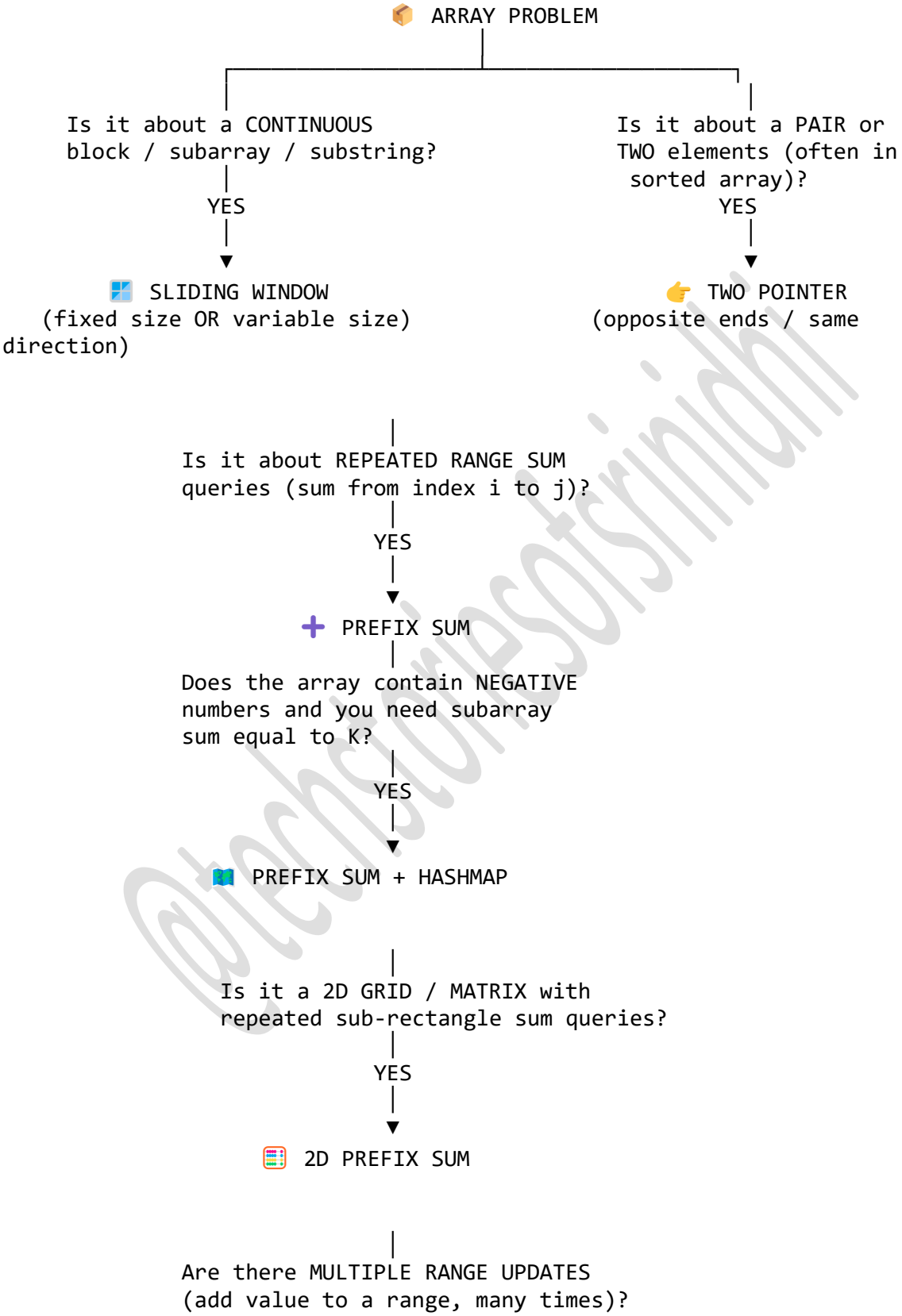
> 📘 **Remember** If you master array patterns deeply, you are not just learning "arrays" — you are building the mental muscle for 60% of all interview problems, across every topic.

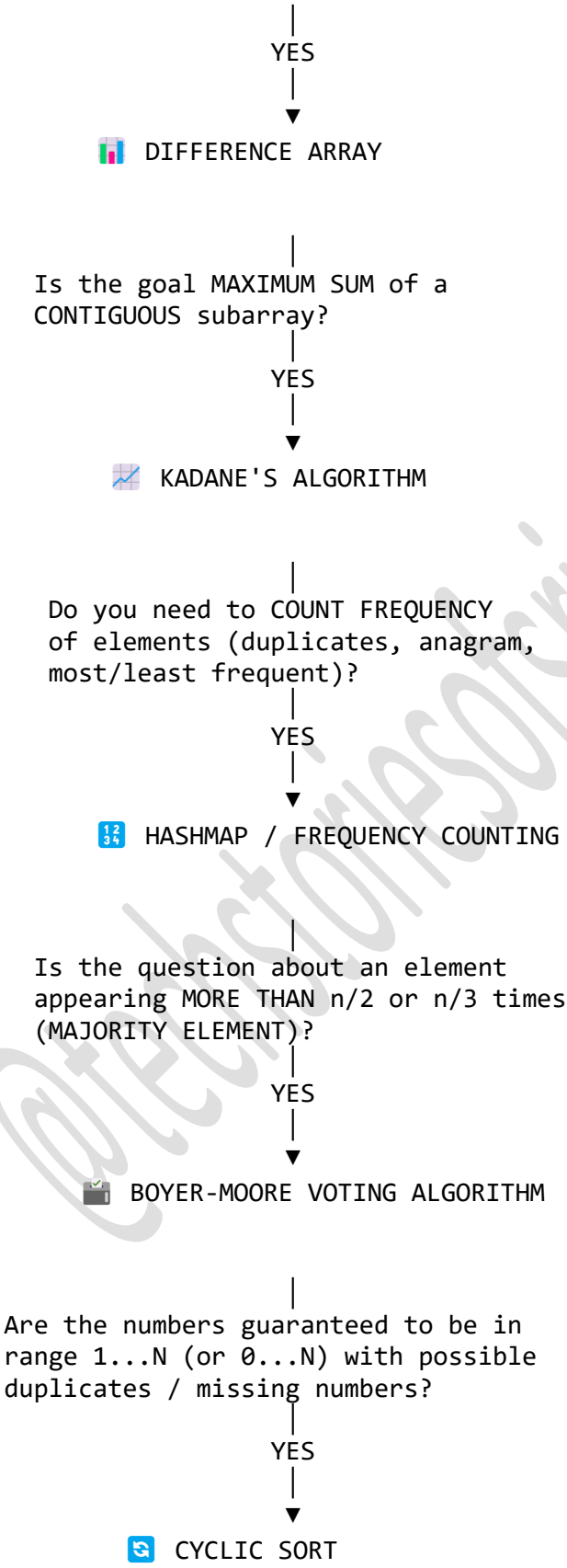
Chapter 2: The Array Pattern Decision Tree

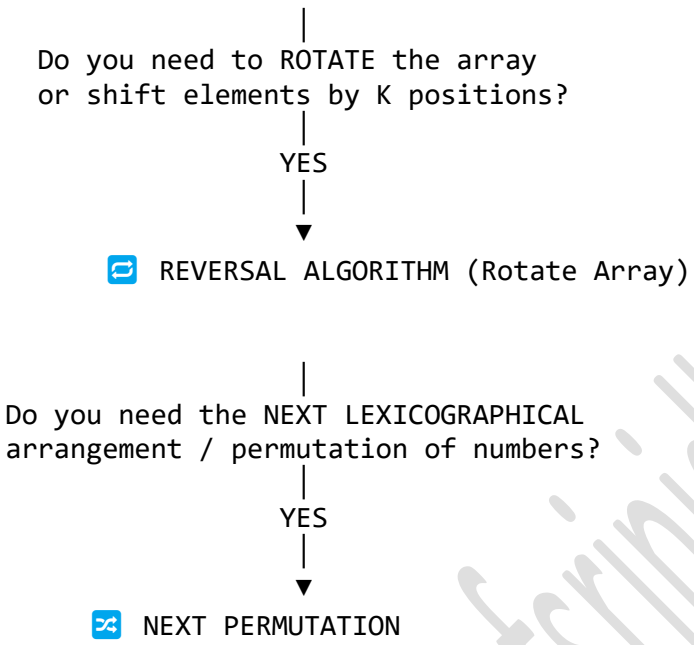
This is the most important page of this entire handbook. Print it. Paste it on your wall. Look at it before every interview.

The goal of this decision tree is simple: **when you read a problem, ask yourself these questions in order, and you will land on the correct pattern almost every time.**

2.1 The Full Decision Tree







2.2 Explaining Every Branch

Let's go branch by branch so this tree actually makes sense and isn't just a picture you memorize blindly.

🪟 Branch 1: Continuous / Subarray / Substring → Sliding Window

If the problem mentions words like *"subarray"*, *"substring"*, *"window of size k"*, *"consecutive"*, or *"contiguous"*, your brain should immediately think **Sliding Window**. Sliding Window helps you avoid recomputing the same overlapping section of an array again and again.

👉 Branch 2: Pair / Two Elements → Two Pointer

If you need to find **two elements** that satisfy a condition (sum, difference, closest value) — especially in a **sorted array** — Two Pointer lets you eliminate brute-force nested loops by moving pointers from both ends intelligently.

➕ Branch 3: Repeated Range Sum → Prefix Sum

If a question asks *"what is the sum from index i to j?"* and this question will be asked **multiple times** (not just once), Prefix Sum precomputes cumulative sums so every query becomes O(1) instead of O(n).

🗺️ Branch 4: Negative Numbers + Subarray Sum = K → Prefix Sum + HashMap

Plain Prefix Sum struggles when numbers can be **negative**, because you can't use a simple two-pointer shrink/expand trick (the sum isn't guaranteed to increase). The fix: store prefix sums in a HashMap so you can instantly check "have I seen `currentSum - K` before?"

🧱 Branch 5: 2D Grid + Repeated Sub-rectangle Queries → 2D Prefix Sum

The same prefix sum idea, but extended to two dimensions — useful for matrix range-sum queries, image processing style problems, and "number of submatrices" questions.

📊 Branch 6: Multiple Range Updates → Difference Array

When you must apply the same "add X to range [i,j]" operation **many times** before reading the final array, updating every index every time is wasteful. Difference Array lets you record just the *start* and *end+1* of each update in O(1), then reconstruct the final array in one pass.

📈 Branch 7: Maximum Sum Contiguous Subarray → Kadane's Algorithm

This is one of the most famous array interview problems in the world. Whenever you hear *"maximum sum subarray"*, Kadane's Algorithm should immediately come to mind — it's a beautiful example of Dynamic Programming hidden inside an "array" question.

🔢 Branch 8: Count / Frequency → HashMap

Anytime you need to know "how many times does X occur", or "which is the most/least frequent element", or check anagram-type conditions, a frequency HashMap (or array-based counting for limited ranges like lowercase letters) is your go-to tool.

🗳️ Branch 9: Majority Element → Boyer-Moore Voting

A very specific but very common interview question: find the element that appears **more than n/2 times**. Boyer-Moore Voting solves this in O(1) space — a huge upgrade over the O(n) space HashMap approach, and interviewers love seeing candidates know this trick.

🔄 Branch 10: Numbers in Range 1…N → Cyclic Sort

If the array is guaranteed to contain numbers within a specific range (like 1 to N), you can place each number at its "correct" index using swaps — turning problems like "find missing number" or "find duplicate number" into elegant O(n) time, O(1) space solutions.

🔁 Branch 11: Rotation → Reversal Algorithm

Rotating an array by K positions has a beautifully elegant O(1) space trick using **three reversals** — no extra array needed.

🔀 Branch 12: Next Arrangement → Next Permutation

When asked to find the "next lexicographically greater permutation" of a sequence, there's a specific well-known algorithm (used internally by `next_permutation` in C++) that you should know cold.

> 💡 **Remember** You will NOT always get a clean signal. Sometimes a problem hides 2 patterns together (e.g., Sliding Window + HashMap, or Prefix Sum +

Kadane). That's normal — real interview questions often **combine** 2 patterns. Once you know each pattern individually, combining them becomes natural.

PART II — THE 12 CORE PATTERNS

Chapter 3: Sliding Window

What is this pattern?

Imagine you have a long train of numbers, and you place a **window** (a frame) over a few of them. As you move the window forward one step at a time, you **add** the new element entering the window and **remove** the element leaving the window — instead of recalculating everything from scratch.

That's it. That's Sliding Window.

In plain English: **Sliding Window helps you avoid recomputation when working with continuous (contiguous) sections of an array or string.**

Think of it like scrolling through Instagram reels — you don't reload the whole app every time; you just load the next reel and unload the previous one. That's exactly how sliding window "slides."

When should I think about this pattern?

Train yourself to notice these **trigger words** in a problem statement:

Trigger Word / Phrase	Why it signals Sliding Window
"Contiguous"	Sliding window only works on continuous sections
"Consecutive"	Same reason — continuity is key
"Subarray"	Subarrays are continuous, unlike subsequences
"Substring"	Same idea, applied to strings
"Window of size K"	Directly tells you it's a fixed window
"Longest / Shortest … satisfying a condition"	Classic variable window signal
"At most K distinct…"	Variable window with a HashMap counter

> ⚠ **Common Mistake** Confusing **subarray** (continuous) with **subsequence** (not necessarily continuous). If elements don't need to stay next to each other, Sliding Window will NOT work — you may need Dynamic Programming instead.

Subpatterns

Sliding Window has **two major flavors**:

Subpattern	Description	Example Problem
Fixed Window	Window size K is given directly	Maximum sum of subarray of size K
Variable Window	Window grows/shrinks based on a condition	Longest substring without repeating characters

Fixed Window — Example Walkthrough

Problem: Find the maximum sum of any subarray of size K.

Brute Force (Level 1): For every starting index, sum up K elements → O(n*k)

Optimal (Level 3) using Sliding Window:

```
windowSum = sum(arr[0:k])
maxSum = windowSum

for i in range(k, len(arr)):
    windowSum += arr[i] - arr[i-k]   # add new, remove old
    maxSum = max(maxSum, windowSum)
```

Time Complexity: **O(n)** instead of O(n*k). We never recompute the whole window — we just "slide" it by adjusting the sum.

Variable Window — Example Walkthrough

Problem: Find the length of the longest substring without repeating characters.

```
seen = set()
left = 0
maxLen = 0

for right in range(len(s)):
    while s[right] in seen:
        seen.remove(s[left])
        left += 1
    seen.add(s[right])
    maxLen = max(maxLen, right - left + 1)
```

Here, the window **grows** by moving `right`, and **shrinks** by moving `left` whenever the condition (no duplicate characters) is violated. This "grow-shrink" motion is the signature of a Variable Sliding Window.

📋 Common Interview Questions & Variations

Variation	Example Question
Fixed size window, find max/min sum	Max sum subarray of size K
Fixed size window, exact condition	Count subarrays of size K with sum ≥ target
Variable window, longest valid	Longest substring without repeating characters
Variable window, shortest valid	Minimum size subarray sum ≥ target
Variable window with HashMap counter	Longest substring with at most K distinct characters
Window + Frequency Matching	Find all anagrams of a string in another string

🧠 Mental Model

When solving a sliding window problem, always ask yourself these 3 questions in order:

1. **Is my window size fixed or variable?**
2. **What state do I need to track inside the window?** (sum, count, frequency map, distinct elements)
3. **When do I shrink the window?** (only for variable window — define the exact "invalid" condition clearly)

> ✅ **Interview Insight** Say this out loud in your interview: *"I'll use two pointers, `left` and `right`, to represent my window. I'll expand `right` to grow the window, and shrink `left` whenever [condition] is violated."* This single sentence instantly tells the interviewer you know the pattern.

⚠️ Common Mistakes

1. Forgetting to shrink the window when the condition breaks (leads to wrong/infinite results).
2. Using sliding window on problems involving **non-contiguous** elements (subsequences) — this pattern does NOT apply there.
3. Recomputing the sum/frequency map from scratch every time the window moves, instead of incrementally updating it.
4. Off-by-one errors when calculating window length: always `right - left + 1`, not `right - left`.
5. Forgetting to update the answer (`maxLen`, `minLen`) **inside** the loop at the correct place.

⏱️ Time Complexity

Aspect	Complexity
Time	O(n) — each element is added and removed from the window at most once
Space	O(1) for fixed sum window, O(k) if using a HashMap/Set for characters

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Maximum Sum Subarray of Size K	Easy	Learn the fixed window basics	Fixed Sliding Window
2	First Negative Number in Every Window of Size K	Easy	Practice maintaining extra state (deque) in fixed window	Fixed Sliding Window
3	Longest Substring Without Repeating Characters	Medium	The most classic variable window question	Variable Sliding Window
4	Minimum Size Subarray Sum	Medium	Practice shrink condition based on sum	Variable Sliding Window
5	Longest Substring with At Most K Distinct Characters	Medium	Combine window with a frequency HashMap	Variable Window + HashMap
6	Find All Anagrams in a String	Medium	Practice frequency matching inside a window	Fixed Window + HashMap
7	Permutation in String	Medium	Similar to anagram matching, builds pattern intuition	Fixed Window + HashMap
8	Fruit Into Baskets	Medium	Disguised "at most 2 distinct" sliding window problem	Variable Window + HashMap
9	Longest Repeating Character Replacement	Hard	Combine window with "max frequency" tracking	Variable Window + HashMap
10	Sliding Window Maximum	Hard	Learn to use a Deque to maintain window maximum in O(n)	Fixed Window + Monotonic Deque
11	Minimum Window Substring	Hard	The hardest classic variable window problem — combines everything	Variable Window + HashMap

---

Chapter 4: Two Pointer

What is this pattern?

Two Pointer is exactly what it sounds like: you use **two index variables** (pointers) that move through the array — either from **opposite ends moving toward each other**, or **both from the start moving at different speeds**. Instead of checking every possible pair with nested loops, you intelligently move the pointers based on a condition, cutting your time complexity dramatically.

Think of it like two people searching a sorted bookshelf from both ends, meeting somewhere in the middle — much faster than one person checking every combination of two books.

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Two Pointer
"Sorted array" + "pair"	Sorted order lets pointers move meaningfully
"Find two numbers that add up to X"	Classic two-sum-in-sorted-array setup
"Remove duplicates in place"	Same-direction two pointer (slow/fast)
"Palindrome check"	Opposite-end two pointer
"Container with most water"	Opposite-end two pointer with greedy shrink
"Merge two sorted arrays"	Same-direction two pointer, one per array

> ⚠️ **Common Mistake** Trying to use Two Pointer on an **unsorted** array when the problem needs order-dependent movement (like Two Sum on unsorted array with pair-sum condition) — you must sort first, OR use a HashMap approach instead if sorting is not allowed (e.g., when you need index positions, not values).

Subpatterns

Subpattern	Description	Example Problem
Opposite Direction	One pointer starts at index 0, other at the end; they move toward each other	Two Sum II (sorted array), Container With Most Water
Same Direction (Slow-Fast)	Both pointers start at index 0, but move at different speeds/conditions	Remove Duplicates from Sorted Array, Move Zeroes

Opposite Direction — Example Walkthrough

Problem: Given a sorted array, find two numbers that add up to a target.

```
left, right = 0, len(arr) - 1

while left < right:
    currentSum = arr[left] + arr[right]
    if currentSum == target:
        return [left, right]
    elif currentSum < target:
        left += 1     # need a bigger sum, move left pointer right
    else:
        right -= 1    # need a smaller sum, move right pointer left
```

Time Complexity: **O(n)** instead of the brute force $O(n^2)$ nested loop.

Same Direction — Example Walkthrough

Problem: Remove duplicates from a sorted array in place.

```
slow = 0
for fast in range(1, len(arr)):
    if arr[fast] != arr[slow]:
        slow += 1
        arr[slow] = arr[fast]

return slow + 1   # new length
```

Here, `slow` marks the "last confirmed unique position," and `fast` scans ahead looking for the next unique value.

Common Interview Questions & Variations

Variation	Example Question
Pair sum in sorted array	Two Sum II
Triplet sum	3Sum, 3Sum Closest
In-place array modification	Remove Duplicates, Move Zeroes
Palindrome verification	Valid Palindrome
Area/Container maximization	Container With Most Water, Trapping Rain Water
Merging	Merge Sorted Array

Mental Model

Ask yourself:

1. **Is the array sorted (or can I sort it without breaking the question)?**
2. **Am I comparing from both ends, or scanning in the same direction at different speeds?**
3. **What decision moves each pointer?** (usually: "if sum too big, shrink from the right; if too small, grow from the left")

> ✅ **Interview Insight** For 3Sum-style problems, always mention: *"I'll fix one element, then use two-pointer on the remaining sorted part for the other two — this reduces the problem from $O(n^3)$ brute force to $O(n^2)$."* This shows you understand how patterns **combine**.

⚠ Common Mistakes

1. Forgetting to **sort** the array first when the problem allows reordering.
2. Not handling **duplicate values** properly in problems like 3Sum (leads to duplicate triplets in the answer).

3. Moving both pointers at the same time incorrectly, skipping potential valid answers.
4. Using Two Pointer when the problem is actually about **subsequences**, where order and position matter differently.
5. Infinite loops caused by forgetting to increment/decrement a pointer in every branch.

Time Complexity

Aspect	Complexity
Time	O(n) for pair problems, $O(n^2)$ for triplet problems (2Sum inside a loop)
Space	O(1) extra space (excluding sort, which is O(log n) to O(n))

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Two Sum II - Input Array Is Sorted	Easy	The foundational two-pointer problem	Opposite Direction
2	Valid Palindrome	Easy	Practice pointer movement with character skipping	Opposite Direction
3	Move Zeroes	Easy	Practice same-direction pointer for in-place shifting	Same Direction
4	Remove Duplicates from Sorted Array	Easy	Classic slow-fast pointer pattern	Same Direction
5	Sort Colors (Dutch National Flag)	Medium	Introduces 3-pointer variation	Three Pointer
6	Container With Most Water	Medium	Teaches greedy pointer-shrinking logic	Opposite Direction
7	3Sum	Medium	Combine sorting + fixed element + two pointer	Fix + Two Pointer
8	3Sum Closest	Medium	Variation focused on “closest” instead of “exact”	Fix + Two Pointer
9	Trapping Rain Water	Hard	One of the most famous FAANG questions	Opposite Direction
10	4Sum	Hard	Extend two-pointer thinking to 4 elements	Fix Fix + Two Pointer

---

Chapter 5: Prefix Sum

What is this pattern?

Prefix Sum is one of the simplest yet most powerful ideas in array problems. The idea: **precompute the cumulative sum up to every index**, so that any range sum query (sum

between index i and j) can be answered instantly, without looping through that range again.

Think of it like a bank statement showing your **running balance** after every transaction. If you want to know how much you spent between day 5 and day 10, you don't re-add every transaction — you just subtract balance(day 4) from balance(day 10).

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Prefix Sum
"Sum of elements between index i and j"	The literal definition of a range sum query
"Multiple queries"	If it's just ONE query, brute force might be fine — but MANY queries screams Prefix Sum
"Equilibrium index"	Needs prefix and suffix sum comparison
"Subarray sum equals K" (without negatives)	Can use prefix sum with a sliding window
"Range update / range query"	Sometimes paired with Difference Array too

> ⚠️ **Common Mistake** Using plain Prefix Sum (without a HashMap) for arrays that contain **negative numbers** when looking for "subarray sum equals K" — this only works cleanly with a sliding window when all numbers are non-negative. For negatives, jump to **Prefix Sum + HashMap** (Chapter 6).

Subpatterns

Subpattern	Description	Example Problem
1D Prefix Sum	Cumulative sum array for range queries	Range Sum Query - Immutable
Prefix + Suffix Sum	Compute sums from both directions	Find Equilibrium/Pivot Index

Example Walkthrough

Problem: Given an array, answer multiple queries of "sum from index i to j".

Brute Force (Level 1): For each query, loop from i to j → O(n) per query, O(n*q) total for q queries.

Optimal (Level 3) using Prefix Sum:

```
prefix = [0] * (len(arr) + 1)
for i in range(len(arr)):
    prefix[i+1] = prefix[i] + arr[i]

def rangeSum(i, j):
    return prefix[j+1] - prefix[i]
```

Now every query is answered in **O(1)**, after an O(n) one-time preprocessing step. This is a massive optimization when there are many queries.

Common Interview Questions & Variations

Variation	Example Question
Static range sum queries	Range Sum Query - Immutable
Pivot/Equilibrium index	Find Pivot Index
Product instead of sum	Product of Array Except Self (uses prefix + suffix product)
Running sum output	Running Sum of 1d Array

Mental Model

Ask yourself:

1. **Will I need the sum of a range MORE THAN ONCE?** If yes, precomputing pays off.
2. **Do I need sum from both left and right?** (prefix AND suffix) — common in "pivot index" style problems.
3. **Is there a HashMap layer needed** because of negative numbers or an exact "sum = K" condition? (→ Chapter 6)

> ✅ **Interview Insight** Say: *"Since we have multiple range-sum queries, I'll precompute a prefix sum array once in O(n), so each query becomes O(1) instead of O(n)."* This single sentence demonstrates strong optimization thinking.

Common Mistakes

1. Off-by-one errors — forgetting that `prefix[i+1]` corresponds to the sum of the first `i+1` elements, not `i`.
2. Not initializing `prefix[0] = 0` (this represents "sum of zero elements").
3. Using Prefix Sum on problems that actually require Prefix Sum + HashMap (subarray sum equals K, with negatives).
4. Forgetting to handle queries where `i = 0` (must not subtract `prefix[-1]`).

Time Complexity

Aspect	Complexity
Time	O(n) to build prefix array, O(1) per query
Space	O(n) for the prefix array

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Running Sum of 1d Array	Easy	Learn to literally build a prefix sum array	1D Prefix Sum
2	Find Pivot Index	Easy	Combine prefix and suffix sum comparison	Prefix + Suffix Sum

#	Question Name	Difficulty	Why solve this?	Expected Pattern
3	Range Sum Query - Immutable	Easy	Classic multiple range-sum-query problem	1D Prefix Sum
4	Product of Array Except Self	Medium	Prefix and suffix PRODUCT instead of sum	Prefix + Suffix Product
5	Contiguous Array	Medium	Combine prefix sum with HashMap for 0/1 balance	Prefix Sum + HashMap
6	Maximum Size Subarray Sum Equals K	Medium	Classic transition problem into Chapter 6	Prefix Sum + HashMap

Chapter 6: Prefix Sum + HashMap

🗺 What is this pattern?

This pattern is an upgrade to plain Prefix Sum. It solves a critical limitation: what happens when the array has **negative numbers**, and you need to find a subarray whose sum equals exactly K?

With negative numbers, a simple sliding window doesn't work because adding more elements doesn't guarantee the sum only increases — it might decrease too. So instead, we use a **HashMap to remember every prefix sum we've seen so far**, along with how many times (or at which index) we've seen it.

The core insight: **if `currentPrefixSum - K` has occurred before, it means there's a subarray ending at the current index whose sum is exactly K.**

Think of it like checking your bank balance: if your balance today is ₹5000, and you remember your balance was ₹3000 at some point in the past, then you know you *earned* exactly ₹2000 in between — without re-adding every transaction.

🚨 When should I think about this pattern?

Trigger Word / Phrase	Why it signals Prefix Sum + HashMap
"Subarray sum equals K"	The most direct signal for this pattern
"Array contains negative numbers"	Rules out simple sliding window
"Count subarrays with sum = K"	Needs frequency counting of prefix sums
"Longest subarray with sum = K"	Needs first-occurrence index of prefix sums

> ⚠ **Common Mistake** Forgetting to initialize the HashMap with `{0: 1}` (or `{0: -1}` for index-based problems) — this represents "a prefix sum of 0 occurring once before we even start," which is crucial for catching subarrays that start from index 0.

Subpatterns

Subpattern	Description	Example Problem
Count subarrays with sum = K	HashMap stores frequency of each prefix sum	Subarray Sum Equals K
Longest subarray with sum = K	HashMap stores the FIRST index where each prefix sum occurred	Maximum Size Subarray Sum Equals K

Example Walkthrough

Problem: Count the number of subarrays whose sum equals K (array may contain negatives).

```
from collections import defaultdict

count = 0
currentSum = 0
prefixCount = defaultdict(int)
prefixCount[0] = 1     # important base case

for num in arr:
    currentSum += num
    if (currentSum - k) in prefixCount:
        count += prefixCount[currentSum - k]
    prefixCount[currentSum] += 1

return count
```

This runs in a single pass — **O(n) time** — compared to the brute force $O(n^2)$ approach of checking every subarray.

Common Interview Questions & Variations

Variation	Example Question
Count subarrays with sum K	Subarray Sum Equals K
Longest subarray with sum K	Maximum Size Subarray Sum Equals K
Subarray sum divisible by K	Subarray Sums Divisible by K (uses modulo + HashMap)
Balanced 0s and 1s	Contiguous Array (treat 0 as -1, then find sum = 0)

Mental Model

Ask yourself:

1. **Does the array contain negative numbers?** If yes, plain sliding window/prefix sum won't reliably work.
2. **Am I counting subarrays, or finding the longest/shortest one?** This decides whether the HashMap stores a **frequency** or a **first-seen index**.

3. **Do I need `prefixSum[i] - prefixSum[j] = K`?** This is the core equation — rearranged as `prefixSum[j] = prefixSum[i] - K`, which is exactly what you look up in the HashMap.

✅ **Interview Insight** Say: *"I'll track running prefix sums in a HashMap. For every new prefix sum, I check if `prefixSum - K` has occurred before — that tells me a valid subarray ending here exists."* This is a very "senior-sounding" explanation that interviewers love.

⚠ Common Mistakes

1. Forgetting the `{0: 1}` base case, causing missed subarrays that start from index 0.
2. Confusing "count" problems (store frequency) with "longest" problems (store first index only, since later occurrences would give shorter subarrays).
3. Not handling negative numbers properly by mistakenly trying a sliding window.
4. Incorrectly computing the lookup key — remember it's `currentSum - K`, not `K - currentSum`.

⏱ Time Complexity

Aspect	Complexity
Time	O(n) — single pass with O(1) HashMap operations
Space	O(n) — to store prefix sums in the HashMap

📝 Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Subarray Sum Equals K	Medium	The definitive problem for this pattern	Prefix Sum + HashMap (count)
2	Continuous Subarray Sum	Medium	Adds a "multiple of K" twist using modulo	Prefix Sum + HashMap (modulo)
3	Subarray Sums Divisible by K	Medium	Reinforces modulo-based prefix sum thinking	Prefix Sum + HashMap (modulo)
4	Contiguous Array	Medium	Clever trick: treat 0 as -1	Prefix Sum + HashMap
5	Maximum Size Subarray Sum Equals K	Medium	Introduces "store first index" variation	Prefix Sum + HashMap (longest)

Chapter 7: 2D Prefix Sum

🧱 What is this pattern?

2D Prefix Sum extends the same idea from Chapter 5, but for a **grid or matrix**. Instead of a single row of numbers, you now have rows AND columns, and you need to answer questions like "what is the sum of all values inside this rectangle?" — potentially many times.

Think of it like a spreadsheet where each cell shows the **total sum of everything above and to the left of it**. Using simple inclusion-exclusion (like an overlapping Venn diagram), you can compute the sum of ANY rectangle in O(1).

When should I think about this pattern?

Trigger Word / Phrase	Why it signals 2D Prefix Sum
"Matrix" or "Grid"	Immediately signals 2D structure
"Sum of a sub-rectangle / sub-matrix"	The literal definition of the pattern
"Multiple range queries on a matrix"	Similar to 1D — many queries = precompute
"Number of submatrices with sum = target"	Combines 2D Prefix Sum with HashMap (advanced)

> ⚠️ **Common Mistake** Getting confused with the inclusion-exclusion formula. Remember: when you add rectangle-above and rectangle-left, you **double-count** the top-left overlapping rectangle, so you must **subtract it once**.

The Core Formula

Given a prefix matrix `P` where `P[i][j]` = sum of all elements from `(0,0)` to `(i-1,j-1)`:

```
P[i][j] = P[i-1][j] + P[i][j-1] - P[i-1][j-1] + matrix[i-1][j-1]
```

To find the sum of a rectangle from `(r1,c1)` to `(r2,c2)`:

```
sum = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]
```

This is basic set theory (inclusion-exclusion) applied to grids — the same logic used in overlapping Venn diagrams.

Common Interview Questions & Variations

Variation	Example Question
Basic 2D range sum queries	Range Sum Query 2D - Immutable
Counting submatrices with target sum	Number of Submatrices That Sum to Target
Matrix block sum	Matrix Block Sum

Mental Model

Ask yourself:

1. **Am I dealing with a fixed matrix and multiple rectangle sum queries?** If yes, build the 2D prefix sum matrix once.
2. **Can I break this problem into "fix two boundaries, then use 1D logic on the rest"?** (Used in advanced problems like counting submatrices with target sum — fix top and bottom rows, then use 1D Prefix Sum + HashMap on columns.)

✅ **Interview Insight** Say: *"I'll extend prefix sum to 2 dimensions using inclusion-exclusion, so any sub-rectangle sum can be computed in O(1) after O(rows × cols) preprocessing."*

⚠ Common Mistakes

1. Forgetting the extra row/column padding (using a `(rows+1) x (cols+1)` prefix matrix avoids messy boundary checks).
2. Getting the inclusion-exclusion signs wrong (a very common silly mistake under interview pressure — practice this formula until it's muscle memory).
3. Not recognizing when a 2D problem can be simplified into repeated 1D Prefix Sum + HashMap logic.

Time Complexity

Aspect	Complexity
Time	O(rows × cols) to build, O(1) per query
Space	O(rows × cols) for the prefix matrix

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Range Sum Query 2D - Immutable	Medium	The foundational 2D prefix sum problem	2D Prefix Sum
2	Matrix Block Sum	Medium	Practice applying the formula with sliding boundaries	2D Prefix Sum
3	Number of Submatrices That Sum to Target	Hard	Combines 2D Prefix Sum with 1D HashMap technique	2D Prefix Sum + HashMap

---

Chapter 8: Difference Array

What is this pattern?

Difference Array is the "opposite" trick of Prefix Sum. Instead of answering **range sum queries**, it efficiently handles **range update operations** — like "add value V to every element from index i to j" — when this needs to happen **many times** before you read the final array.

Think of it like editing a spreadsheet: instead of manually updating every single cell in a range every time (slow), you just leave a "start increasing here" marker and a "stop increasing here" marker. At the end, one pass converts these markers into the actual final values.

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Difference Array
"Add value to range [i, j], multiple times"	The literal definition of the pattern
"Multiple bookings / multiple range updates, then read final result"	Common in scheduling-style problems
"Range increment, then query final array"	Perfect match

> ⚠️ **Common Mistake** Using Difference Array for problems that need the array's value **during** the updates (not just at the end). Difference Array only works well when you can apply ALL updates first, and read the final result LAST.

The Core Idea

Given a `diff` array of the same size (plus 1 extra slot):

```
def applyRangeUpdate(diff, i, j, value):
    diff[i] += value
    if j + 1 < len(diff):
        diff[j + 1] -= value

# After all updates, reconstruct the final array:
result = [0] * n
result[0] = diff[0]
for i in range(1, n):
    result[i] = result[i-1] + diff[i]
```

Each update takes **O(1)**, no matter how large the range is. The final reconstruction takes **O(n)** — one single pass, regardless of how many updates were applied.

Common Interview Questions & Variations

Variation	Example Question
Basic range increment	Range Addition
Booking/interval overlap counting	Corporate Flight Bookings
Car pooling capacity check	Car Pooling

Mental Model

Ask yourself:

1. **Do I have MANY range-update operations, and only need the FINAL array at the end?** If yes, Difference Array saves huge time.

2. **Am I trying to read the array's value in the MIDDLE of applying updates?** If yes, Difference Array is NOT the right tool — go back to direct simulation or a different approach.

✅ **Interview Insight** Say: *"Since we have multiple range updates but only need the final state, I'll use a difference array — marking the start and end+1 of each range in O(1), then reconstructing the final array in a single O(n) pass."*

⚠ Common Mistakes

1. Forgetting to subtract the value at index `j+1` (this is what "cancels" the addition after the range ends).
2. Off-by-one errors on array boundaries — always allocate `diff` with one extra slot to safely handle `j+1 == n`.
3. Trying to query intermediate states, which Difference Array does not support directly.

Time Complexity

Aspect	Complexity
Time	O(1) per update, O(n) total to reconstruct the final array
Space	O(n) for the difference array

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Range Addition	Medium	The direct, textbook definition of this pattern	Difference Array
2	Corporate Flight Bookings	Medium	Real-world styled range-update problem	Difference Array
3	Car Pooling	Medium	Combine difference array with capacity-checking logic	Difference Array

---

Chapter 9: Kadane's Algorithm

What is this pattern?

Kadane's Algorithm answers one of the most famous interview questions in existence: **"Find the contiguous subarray with the maximum sum."** It is secretly a **Dynamic Programming** problem disguised as an "array" problem — which is exactly why it's such a favorite among interviewers.

The core insight: **at every index, you make a simple decision — should I extend the previous subarray, or start a brand new subarray from here?** You extend the previous subarray if it's still contributing positively; otherwise, starting fresh gives a better result.

Think of it like a stock trader tracking cumulative profit: if your running profit ever drops below zero, you "reset" and start counting fresh from the next day, because carrying negative baggage only hurts your future sum.

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Kadane's Algorithm
"Maximum sum subarray"	The exact definition of the problem
"Maximum sum contiguous"	Same signal, reworded
"Best time to buy and sell stock" (single transaction)	Secretly the same logic as Kadane's
"Maximum product subarray"	A twist on Kadane's — must track both max AND min (because of negative × negative = positive)

> ⚠️ **Common Mistake** Forgetting to handle an array made **entirely of negative numbers**. In that case, the "maximum subarray sum" is simply the largest single negative number — not zero. Beginners often incorrectly initialize their running sum to 0, which breaks this edge case.

Subpatterns

Subpattern	Description	Example Problem
Maximum Sum Kadane	Standard version — track max running sum	Maximum Subarray
Maximum Product Kadane	Track both max AND min running product (due to sign flips)	Maximum Product Subarray
Circular Kadane	Handle subarrays that "wrap around" the array	Maximum Sum Circular Subarray

Example Walkthrough

Problem: Find the maximum sum of a contiguous subarray.

```
maxSum = arr[0]
currentSum = arr[0]

for i in range(1, len(arr)):
    currentSum = max(arr[i], currentSum + arr[i])
    maxSum = max(maxSum, currentSum)

return maxSum
```

The key line is `currentSum = max(arr[i], currentSum + arr[i])`. This says: *"Either I extend the existing subarray by adding the current element, OR I abandon the past and start fresh from the current element — whichever gives a bigger sum."*

Time Complexity: **O(n)** — a single pass, compared to the brute force $O(n^2)$ approach of checking every possible subarray.

📋 Common Interview Questions & Variations

Variation	Example Question
Standard max sum	Maximum Subarray
Max product (sign-flip tracking)	Maximum Product Subarray
Circular array version	Maximum Sum Circular Subarray
Single buy-sell stock	Best Time to Buy and Sell Stock

🧠 Mental Model

Ask yourself at every index:

1. **"Does continuing my current subarray still help me, or is it dragging me down?"**
2. **"Should this update my global maximum?"**

> ✅ **Interview Insight** Say: *"At each index, I decide whether to extend the current subarray or start fresh — this is a classic 1D dynamic programming recurrence disguised as an array problem, and it runs in O(n) time with O(1) space."* Mentioning the DP connection is a strong signal of depth.

⚠️ Common Mistakes

1. Initializing `maxSum = 0` instead of `arr[0]` — breaks all-negative arrays.
2. Forgetting that for "maximum product," you must track both the running **max** and running **min**, because multiplying two negative numbers can flip a small negative product into a large positive one.
3. Confusing "maximum subarray sum" (contiguous) with "maximum subsequence sum" (not necessarily contiguous) — these are very different problems.

⏱️ Time Complexity

Aspect	Complexity
Time	O(n) — single pass
Space	O(1)

📝 Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Maximum Subarray	Medium	The definitive Kadane's Algorithm problem	Kadane's Algorithm
2	Best Time to Buy and Sell Stock	Easy	Same underlying logic, different story	Kadane's Algorithm (variant)
3	Maximum Product Subarray	Medium	Adds the max/min tracking twist	Kadane's Algorithm (product)
4	Maximum Sum Circular Subarray	Medium	Adds circular array wraparound logic	Kadane's Algorithm (circular)

Chapter 10: Frequency Counting

What is this pattern?

Frequency Counting is one of the most versatile patterns in all of DSA. The idea is simple: **use a HashMap (or a fixed-size array, for limited character sets) to count how many times each element appears.** Once you have this frequency data, a huge range of problems become trivial to solve.

Think of it like taking attendance in a classroom — once you have a tally of how many times each name appears, you can instantly answer questions like "who is present most often?" or "is anyone missing?"

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Frequency Counting
"How many times does X occur?"	Direct frequency question
"Most / least frequent element"	Needs a frequency map, then a max/min scan
"Anagram"	Needs character frequency comparison
"Duplicate elements"	Needs a seen/frequency map
"Group elements by some property"	Needs a HashMap of lists (grouping)

> ⚠ **Common Mistake** Using a HashMap when the input is restricted to a small, known range (like lowercase English letters). In that case, a fixed-size array of 26 slots is faster and uses less overhead than a HashMap.

Subpatterns

Subpattern	Description	Example Problem
Simple Frequency Map	Count occurrences of each element	First Unique Character
Frequency Comparison	Compare two frequency maps	Valid Anagram
Group by Key	Use frequency/signature as a grouping key	Group Anagrams
Top-K Frequency	Combine frequency count with sorting/heap	Top K Frequent Elements

Example Walkthrough

Problem: Check if two strings are anagrams of each other.

```
from collections import Counter

def isAnagram(s, t):
    return Counter(s) == Counter(t)
```

Or, without using a library, using a fixed 26-size array (since only lowercase letters are involved):

```
def isAnagram(s, t):
    if len(s) != len(t):
        return False
    count = [0] * 26
    for ch in s:
        count[ord(ch) - ord('a')] += 1
    for ch in t:
        count[ord(ch) - ord('a')] -= 1
    return all(c == 0 for c in count)
```

Common Interview Questions & Variations

Variation	Example Question
Basic frequency counting	First Unique Character in a String
Frequency comparison	Valid Anagram
Grouping by frequency signature	Group Anagrams
Top-K frequency + sorting/heap	Top K Frequent Elements
Frequency + sliding window	Find All Anagrams in a String

Mental Model

Ask yourself:

1. **Do I need to know how many times something appears?** → Build a frequency map.
2. **Is the input range small and known (like lowercase letters)?** → Use a fixed-size array instead of a HashMap for speed.
3. **Am I comparing frequency patterns between two inputs?** → Build two frequency maps (or use one map with +1/-1 logic) and compare.

✅ **Interview Insight** Say: *"Since the alphabet is limited to lowercase letters, I can use a fixed 26-size array instead of a HashMap — this keeps space constant instead of growing with input size."* Interviewers love hearing this kind of micro-optimization awareness.

⚠ Common Mistakes

1. Using a HashMap when a simple fixed-size array would be faster and simpler.
2. Forgetting to handle case sensitivity or non-alphabetic characters.
3. Not considering that two frequency maps must match **exactly** (both same keys AND same counts) for anagram-type problems.

Time Complexity

Aspect	Complexity
Time	O(n) to build the frequency map
Space	O(k) where k is the number of distinct elements (O(1) if a fixed alphabet is used)

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	First Unique Character in a String	Easy	Practice basic frequency counting	Frequency Counting
2	Valid Anagram	Easy	Practice frequency comparison	Frequency Counting
3	Contains Duplicate	Easy	Practice using a Set/HashMap for presence checking	Frequency Counting
4	Group Anagrams	Medium	Practice using frequency signature as a HashMap key	Frequency Counting (grouping)
5	Top K Frequent Elements	Medium	Combine frequency counting with heap/bucket sort	Frequency Counting + Heap

---

Chapter 11: Boyer-Moore Voting Algorithm

What is this pattern?

Boyer-Moore Voting is a clever, almost magical algorithm used to find the **majority element** — an element that appears **more than n/2 times** in an array — using only **O(1) extra space**. No HashMap, no sorting, just a simple counter and a candidate variable.

Think of it like a tug-of-war: every time you see the current "candidate" element, you pull the rope in its favor (+1). Every time you see a different element, you pull the rope against it (-1). If the rope count ever hits zero, you "give up" on the current candidate and pick a fresh one. Because the majority element appears more than half the time, it always "wins" the tug-of-war by the end.

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Boyer-Moore Voting
"Appears more than n/2 times"	The exact definition of Majority Element
"Appears more than n/3 times"	Extended version needing 2 candidates
"Find the dominant / majority element"	Direct signal
"O(1) space" + "find frequent element"	Space constraint rules out HashMap approach

> ⚠️ **Common Mistake** Assuming a majority element always exists. Many variations of this problem (like "more than n/3 times") ask you to find candidates that **might** be majority elements, then require a **second pass** to verify their actual count.

Subpatterns

Subpattern	Description	Example Problem
Single Majority (n/2)	One candidate, one counter	Majority Element
Extended Majority (n/3)	Two candidates, two counters (since at most 2 elements can appear more than n/3 times)	Majority Element II

Example Walkthrough

Problem: Find the element that appears more than n/2 times.

```
candidate = None
count = 0

for num in arr:
    if count == 0:
        candidate = num
    count += 1 if num == candidate else -1

return candidate
```

Time Complexity: **O(n)** with **O(1) space** — a massive improvement over the O(n) space HashMap-based frequency counting approach.

> 📘 **Remember** This works because the majority element appears more than half the time. Even in the worst-case cancellation pattern, it cannot be fully "cancelled out" by all the other elements combined.

Common Interview Questions & Variations

Variation	Example Question
Majority element (n/2)	Majority Element
Majority elements (n/3)	Majority Element II

Variation	Example Question
Verify candidate with second pass	Majority Element (verification step in edge cases)

Mental Model

Ask yourself:

1. **Is the question asking for an element appearing more than n/2 (or n/3) times?** This is a very specific and recognizable phrasing.
2. **Do I need O(1) space?** If the interviewer explicitly asks for constant space, Boyer-Moore is almost certainly the expected answer instead of a HashMap.
3. **Do I need a verification pass?** For n/3 problems, always do a second pass to confirm the final candidates actually meet the frequency requirement (since the algorithm only finds *potential* candidates).

> ✅ **Interview Insight** Say: *"Since we need O(1) space, I'll use Boyer-Moore Voting — treating this like a cancellation game where the majority element can never be fully cancelled out because it appears more than half the time."*

Common Mistakes

1. Forgetting the verification pass for the "more than n/3" variation, which can lead to false positives.
2. Assuming the algorithm works if NO majority element actually exists (the basic version assumes one is guaranteed by the problem).
3. Confusing this with simple frequency counting (which uses O(n) space) when O(1) space is explicitly required.

Time Complexity

Aspect	Complexity
Time	O(n) — one pass to find candidate(s), optionally a second pass to verify
Space	O(1)

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Majority Element	Easy	The foundational Boyer-Moore problem	Boyer-Moore Voting
2	Majority Element II	Medium	Extend to two candidates and add verification	Boyer-Moore Voting (extended)

Chapter 12: Cyclic Sort

What is this pattern?

Cyclic Sort is a specialized pattern that only applies when the array contains numbers within a **known, limited range** — typically **1 to N** (or 0 to N-1). The idea: **place every number at its "correct" index by swapping**, so that `arr[i] = i + 1` (or similar). Once numbers are correctly placed, finding missing/duplicate numbers becomes trivial with a single pass.

Think of it like organizing numbered lockers: locker #1 should hold key #1, locker #2 should hold key #2, and so on. If you find a key in the wrong locker, you swap it to its correct locker — and repeat until everything is in place.

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Cyclic Sort
"Numbers from 1 to N"	The classic range signal
"Find the missing number"	Very common Cyclic Sort application
"Find the duplicate number"	Same range-based signal
"Find all missing numbers"	Extended version
"In-place, O(1) space" + "range 1 to N"	Strongly hints at Cyclic Sort instead of HashMap

> ⚠ **Common Mistake** Trying to use Cyclic Sort on arrays where numbers are **not** guaranteed to be within a specific range — this pattern **only** works when there's a clear index-value relationship to exploit.

The Core Idea

```
i = 0
while i < len(arr):
    correctIndex = arr[i] - 1     # assuming range is 1 to N
    if arr[i] != arr[correctIndex]:
        arr[i], arr[correctIndex] = arr[correctIndex], arr[i]
    else:
        i += 1
```

After this loop, every number that belongs in the array is placed at its correct index. Any index `i` where `arr[i] != i + 1` reveals a **missing number** (`i + 1`) or points to a **duplicate**.

Example: Finding the missing number in an array of size N containing numbers 0 to N.

```
i = 0
while i < len(arr):
    correctIndex = arr[i]
    if arr[i] < len(arr) and arr[i] != arr[correctIndex]:
        arr[i], arr[correctIndex] = arr[correctIndex], arr[i]
    else:
        i += 1

for i in range(len(arr)):
    if arr[i] != i:
        return i

return len(arr)
```

📋 Common Interview Questions & Variations

Variation	Example Question
Find missing number	Missing Number
Find duplicate number	Find the Duplicate Number
Find all missing numbers	Find All Numbers Disappeared in an Array
Find all duplicate numbers	Find All Duplicates in an Array
Find the first missing positive	First Missing Positive

🧠 Mental Model

Ask yourself:

1. **Are the numbers guaranteed to be within a specific range (like 1 to N, or 0 to N-1)?** This is the #1 signal.
2. **Can I use the VALUE of an element as a hint for its CORRECT INDEX?** If yes, Cyclic Sort applies.
3. **After placing everything correctly, what does a "mismatch" at index i tell me?** (usually reveals missing/duplicate numbers)

> ✅ **Interview Insight** Say: *"Since the numbers are guaranteed to be in the range 1 to N, I can place each number at its correct index using swaps — this gives O(n) time and O(1) extra space, better than using a HashMap which needs O(n) space."*

⚠️ Common Mistakes

1. Using this pattern on arrays without a clear range guarantee — it simply won't work correctly.
2. Writing an infinite loop by forgetting to increment `i` in the "already correctly placed" branch.
3. Off-by-one errors between 1-indexed and 0-indexed range assumptions.

4. Forgetting to handle duplicate values properly (checking `arr[i] != arr[correctIndex]` before swapping prevents infinite loops).

Time Complexity

Aspect	Complexity
Time	O(n) — each element is swapped at most once into its correct position
Space	O(1) — sorting happens in place

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Missing Number	Easy	The foundational cyclic sort problem	Cyclic Sort
2	Find All Numbers Disappeared in an Array	Easy	Extend to finding multiple missing numbers	Cyclic Sort
3	Find the Duplicate Number	Medium	Classic FAANG favorite — multiple solving approaches	Cyclic Sort
4	Find All Duplicates in an Array	Medium	Practice marking visited indices using sign-flipping	Cyclic Sort (variant)
5	First Missing Positive	Hard	One of the hardest and most famous array interview questions	Cyclic Sort

---

Chapter 13: Rotate Array

What is this pattern?

Rotating an array means shifting every element by K positions, wrapping around from the end back to the beginning. The brute force way uses an **extra array** — but there's an elegant trick that does it **in-place** using **three reversals**, needing zero extra space.

Think of it like flipping segments of a rope: reverse the whole rope, then reverse the two segments individually — and somehow, the rope ends up perfectly rotated!

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Reversal-based Rotation
"Rotate the array by K positions"	Direct signal
"Shift elements to the right/left"	Same idea, different wording
"In-place, O(1) extra space"	Rules out the simple "copy to new array" approach

> ⚠ **Common Mistake** Forgetting to take `k % n` first when k is larger than the array length — rotating by n positions brings the array back to its original state, so any k larger than n has redundant full rotations.

The Core Idea (Three Reversals)

Problem: Rotate an array to the right by K positions.

```
def reverse(arr, start, end):
    while start < end:
        arr[start], arr[end] = arr[end], arr[start]
        start += 1
        end -= 1

def rotate(arr, k):
    n = len(arr)
    k = k % n
    reverse(arr, 0, n - 1)        # reverse the whole array
    reverse(arr, 0, k - 1)        # reverse the first k elements
    reverse(arr, k, n - 1)        # reverse the remaining elements
```

Walkthrough with an example: `arr = [1,2,3,4,5,6,7], k = 3`

1. Reverse whole array → `[7,6,5,4,3,2,1]`
2. Reverse first `k=3` elements → `[5,6,7,4,3,2,1]`
3. Reverse remaining elements → `[5,6,7,1,2,3,4]`

Final Result: `[5,6,7,1,2,3,4]` — exactly the array rotated right by 3! ✅

Common Interview Questions & Variations

Variation	Example Question
Rotate right by K	Rotate Array
Rotate left by K	Left Rotation (same idea, reversed direction)
Rotate a 2D matrix	Rotate Image (uses transpose + reverse)

Mental Model

Ask yourself:

1. **Do I need to rotate in-place with O(1) space?** If yes, the three-reversal trick is the expected optimal answer.
2. **Have I taken `k % n` first?** Always normalize k before doing anything else.
3. **Am I rotating right or left?** The direction changes which segments you reverse first.

✅ **Interview Insight** Say: *"I'll normalize k using k % n to avoid redundant full rotations, then use the reversal trick — reverse the whole array, then reverse each of the two segments — to rotate in-place with O(1) extra space."*

Common Mistakes

1. Forgetting `k = k % n`, causing unnecessary extra rotations or index errors when k > n.

2. Reversing the segments in the wrong order (must reverse the WHOLE array first, then the two parts).
3. Off-by-one errors in the reverse() helper's start/end boundaries.

Time Complexity

Aspect	Complexity
Time	O(n) — three linear passes total
Space	O(1) — done entirely in place

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Rotate Array	Medium	The definitive rotation problem — practice all 3 approaches (extra array, cyclic replacement, reversal)	Reversal Algorithm
2	Rotate Image	Medium	Extend rotation thinking to a 2D matrix (90-degree rotation)	Transpose + Reverse
3	Rotate List (Linked List)	Medium	See how the same rotation concept applies beyond arrays	Rotation Logic

---

Chapter 14: Next Permutation

What is this pattern?

Next Permutation finds the **next lexicographically greater arrangement** of a sequence of numbers. If no such arrangement exists (the sequence is in fully descending order), it rearranges the numbers into the **lowest possible order** (fully ascending).

Think of it like counting in a strange numeral system where digits are the actual array elements — "123" becomes "132", which becomes "213", and so on, until you reach "321" (the maximum), after which it wraps back around to "123."

When should I think about this pattern?

Trigger Word / Phrase	Why it signals Next Permutation
"Next permutation"	Direct, unambiguous signal
"Next lexicographical arrangement"	Same idea, formal wording
"Rearrange to the next greater number using the same digits"	Common alternate phrasing

> ⚠ **Common Mistake** Trying to generate ALL permutations and then searching for the next one — this is a massive waste of time (O(n!) instead of O(n)). There is a specific, elegant O(n) algorithm for this exact problem.

The Core Algorithm (4 Steps)

Problem: Given an array of numbers, rearrange it into the next lexicographically greater permutation.

Step 1: Scan from the right, find the first index `i` where `arr[i] < arr[i+1]` (the first "descent point" from the right — this is called the **pivot**).

Step 2: If no such index exists, the array is in fully descending order — simply reverse the entire array (this gives the smallest permutation) and stop.

Step 3: Otherwise, scan from the right again to find the smallest element `arr[j]` that is **greater than** `arr[i]`, and swap `arr[i]` and `arr[j]`.

Step 4: Reverse everything after index `i` (this portion is currently in descending order, and reversing it gives the smallest possible arrangement for that suffix).

```
def nextPermutation(arr):
    n = len(arr)
    i = n - 2

    # Step 1: find the pivot
    while i >= 0 and arr[i] >= arr[i+1]:
        i -= 1

    if i >= 0:
        # Step 3: find the element just larger than arr[i]
        j = n - 1
        while arr[j] <= arr[i]:
            j -= 1
        arr[i], arr[j] = arr[j], arr[i]

    # Step 4: reverse the suffix (handles Step 2 case too when i == -1)
    left, right = i + 1, n - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
```

Walkthrough with an example: `arr = [1,2,3]`

1. Pivot search: `arr[1]=2 < arr[2]=3` → pivot index `i = 1`
2. Find smallest element greater than `arr[1]=2` from the right → `arr[2]=3`
3. Swap → `[1,3,2]`
4. Reverse suffix after index 1 (just one element, no change) → **Final: `[1,3,2]`** ✅

Common Interview Questions & Variations

Variation	Example Question
Standard next	Next Permutation

Variation	Example Question
permutation	
Previous permutation	Previous Permutation (mirror logic)
Kth permutation sequence	Permutation Sequence (uses factorial number system, different approach)

Mental Model

Ask yourself:

1. **Am I looking for the very NEXT arrangement, not all arrangements?** This distinguishes it from brute-force permutation generation.
2. **Have I correctly identified the "pivot" — the first descent from the right?**
3. **Did I reverse the suffix at the end, to make it the smallest possible arrangement?**

> ✅ **Interview Insight** Say: *"I'll find the rightmost ascending pair to identify the pivot, swap it with the next larger element from the suffix, then reverse the suffix to get the smallest possible arrangement — this runs in O(n) time and O(1) space, compared to O(n!) if I tried generating all permutations."*

⚠ Common Mistakes

1. Forgetting to check the "no pivot found" case (fully descending array) — must reverse the entire array in this case.
2. Swapping with the WRONG element in Step 3 — it must be the smallest element greater than `arr[i]`, not just any greater element.
3. Forgetting Step 4 (reversing the suffix) — without this, the result won't be the smallest possible next arrangement.

Time Complexity

Aspect	Complexity
Time	O(n) — a few linear scans
Space	O(1) — done in place

Practice Questions (Easy → Hard)

#	Question Name	Difficulty	Why solve this?	Expected Pattern
1	Next Permutation	Medium	The definitive problem for this exact pattern	Next Permutation Algorithm
2	Permutation Sequence	Hard	A related but distinct problem using factorial number system	Mathematical Permutation
3	Next Greater Element III	Medium	Applies the same next-permutation logic to digits of a number	Next Permutation Algorithm

PART III — THINKING LIKE A FAANG ENGINEER

Chapter 15: Array Optimization Playbook

Every optimization in array problems follows a small number of repeatable "upgrade paths." Once you recognize which upgrade path applies, half your interview is already solved. Let's go through each one.

15.1 Nested Loops → Two Pointer / HashMap

The Problem Signal: You've written a brute-force solution with two nested loops ($O(n^2)$) to check every pair of elements.

The Upgrade: Ask "can I sort the array (Two Pointer) or remember what I've already seen (HashMap) to avoid re-scanning?"

Before (Nested Loop)	After (Optimized)
Check every pair for sum = target → $O(n^2)$	Sort + Two Pointer → $O(n \log n)$, or HashMap → $O(n)$
Check every pair for closest values	Sort + Two Pointer → $O(n \log n)$

Example:

```
# BEFORE: O(n²)
for i in range(n):
    for j in range(i+1, n):
        if arr[i] + arr[j] == target:
            return True

# AFTER: O(n) using HashMap
seen = set()
for num in arr:
    if target - num in seen:
        return True
    seen.add(num)
```

15.2 Repeated Calculations → Prefix Sum

The Problem Signal: You're re-summing the same range of the array over and over across multiple queries.

The Upgrade: Precompute a Prefix Sum array ONCE, so every future query becomes $O(1)$.

Before	After
Loop through range for every query → $O(n)$ per query	Precompute prefix sums → $O(1)$ per query after $O(n)$ setup

15.3 Multiple Passes → Single Pass

The Problem Signal: Your solution loops through the array 2–3 separate times to gather different pieces of information.

The Upgrade: Ask "can I gather all the information I need in ONE single pass, using a few extra variables?"

Example: Finding both the minimum and maximum in an array.

```
# BEFORE: Two passes
minVal = min(arr)
maxVal = max(arr)

# AFTER: One pass
minVal, maxVal = arr[0], arr[0]
for num in arr:
    minVal = min(minVal, num)
    maxVal = max(maxVal, num)
```

While both are technically O(n), in interviews, showing you can combine logic into a single pass demonstrates strong control over your solution — especially important when each "pass" involves expensive work.

15.4 Extra Space → In-Place

The Problem Signal: You're creating a brand new array/HashMap to store transformed results, when the original array could be reused.

The Upgrade: Ask "can I overwrite the original array using clever index tricks (like Cyclic Sort, or two-pointer swapping) instead of allocating new space?"

Before	After
Create a new array for rotation → O(n) space	Use 3 reversals → O(1) space
Use a HashMap to detect missing numbers 1 to N → O(n) space	Use Cyclic Sort → O(1) space
Create a new array to remove duplicates	Use slow-fast two-pointer in place → O(1) space

> 💡 **Remember** Interviewers explicitly ask "can you do this with O(1) space?" MORE OFTEN than you'd expect. Always have an in-place alternative ready for common problems.

15.5 The Universal Optimization Checklist

Before finalizing any array solution, run through this checklist:

#	Question	If YES, consider…
1	Am I using nested loops to check pairs?	Two Pointer or

#	Question	If YES, consider…
		HashMap
2	Am I repeating range-sum calculations?	Prefix Sum
3	Does my array have negative numbers and I need subarray sum = K?	Prefix Sum + HashMap
4	Am I making multiple passes for related information?	Combine into a single pass
5	Am I creating unnecessary new arrays/objects?	In-place modification
6	Is the range of numbers limited (1 to N)?	Cyclic Sort
7	Do I need a majority element with O(1) space?	Boyer-Moore Voting

Chapter 16: Top 20 Array Interview Mistakes

Even strong students lose marks because of small, repeatable mistakes — not because they don't know the pattern. Here are the 20 most common ones, based on real interview feedback patterns.

#	Mistake	Why it Hurts You	Fix
1	Jumping straight to code without explaining approach	Looks like memorization, not understanding	Always explain brute force → optimal out loud first
2	Not clarifying constraints before coding	May solve the wrong version of the problem	Ask about array size, duplicates, negative numbers, sorted or not
3	Ignoring edge cases (empty array, single element)	Causes runtime errors in front of the interviewer	Always mention & test edge cases explicitly
4	Off-by-one errors in loops	Small bugs that are hard to spot under pressure	Practice writing loop boundaries carefully; test manually
5	Confusing subarray with subsequence	Leads to applying the wrong pattern entirely	Always confirm: "contiguous" = subarray, "not necessarily contiguous" = subsequence
6	Forgetting to handle duplicate values	Wrong answers in Two Pointer / Cyclic Sort problems	Explicitly consider duplicate-handling logic
7	Using extra space when O(1) was expected	Loses "optimal" points even if answer is correct	Always mention the space-optimized alternative
8	Not stating time/space complexity after coding	Interviewer has to ask, which looks less proactive	Always state Big-O immediately after finishing the code

#	Mistake	Why it Hurts You	Fix
9	Forgetting the base case in Prefix Sum + HashMap (`{0:1}`)	Missed valid subarrays starting at index 0	Always initialize the HashMap with the zero-sum base case
10	Assuming a majority element always exists	Wrong answers when the array has no true majority	Confirm with interviewer whether existence is guaranteed
11	Not normalizing `k = k % n` in rotation problems	Index errors or wasted extra rotations	Always normalize k before rotating
12	Using Sliding Window on non-contiguous problems	Fundamentally wrong approach	Confirm "contiguous" requirement before choosing this pattern
13	Forgetting to shrink the window when a condition breaks	Incorrect longest/shortest answers	Always define a clear "invalid" condition and shrink accordingly
14	Mixing up "count" vs "index" HashMap storage	Wrong answers in count vs longest-subarray problems	Store frequency for "count" problems, first-index for "longest" problems
15	Not considering integer overflow (in languages like Java/C++)	Can silently produce wrong answers	Mention overflow-safe practices when relevant
16	Silence while thinking	Interviewer can't follow your reasoning	Narrate your thought process, even mid-struggle
17	Giving up after one failed approach	Looks like lack of resilience	Say "this approach won't work because X, let me try Y"
18	Not testing the code with a sample input after writing it	Bugs go unnoticed until the interviewer points them out	Always dry-run your code with a small example before saying "done"
19	Over-engineering a simple problem	Wastes time; can look like you're overcomplicating	Match your solution's complexity to the problem's actual difficulty
20	Not asking about input size (N)	Wrong complexity choice (e.g., $O(n^2)$ might be fine for small N)	Always ask about expected input size to judge acceptable complexity

⚠️ **Common Mistake** Many candidates know the RIGHT pattern but lose points on execution — small bugs, unclear communication, or skipping edge cases. Technical knowledge alone is not enough; **communication and precision** matter just as much.

Chapter 17: FAANG Interview Tips

17.1 How Interviewers Actually Think

Interviewers are not trying to "trick" you. Most FAANG interviewers follow an internal scoring rubric that typically includes:

Evaluation Area	What They're Checking
Problem-solving	Did you break down the problem logically?
Coding ability	Is your code clean, correct, and bug-free?
Communication	Can you explain your thinking clearly?
Optimization	Did you improve from brute force to optimal?
Testing	Did you verify your solution with examples?

> 📘 **Remember** A candidate who solves the problem in a "so-so" way but communicates excellently often scores HIGHER than a silent candidate who solves it perfectly. Communication is not optional — it is graded.

17.2 How to Identify Patterns Quickly

Follow this 4-step process every single time you read a new problem:

1. **Read the problem twice.** Once for the story, once for the constraints.
2. **Identify the keywords.** ("contiguous," "sorted," "range sum," "majority," "1 to N," etc.)
3. **Match keywords to the Decision Tree (Chapter 2).** This narrows your options to 1-2 likely patterns.
4. **State your hypothesis out loud.** "This looks like a Sliding Window problem because we need a contiguous subarray satisfying a sum condition." Even if you're wrong, the interviewer will often nudge you in the right direction.

17.3 How to Communicate During Interviews

Use this simple 5-step communication framework in every interview:

Step	What to Say
1. Clarify	"Can I confirm — is the array sorted? Can it have negative numbers? What's the expected size of N?"
2. Brute Force	"The brute force approach would be… this gives $O(n^2)$ time."
3. Optimize	"I can improve this using [pattern name], bringing it down to O(n)."
4. Code	"Let me code this now, I'll think out loud as I go."
5. Test	"Let me trace through this with an example to confirm it works."

> ✅ **Interview Insight** Interviewers often ask "can you optimize this further?" even when your solution is already optimal — this is sometimes just to test if

you'll second-guess a correct answer. It's okay to confidently say: *"I believe this is already optimal at O(n) time and O(1) space, because we need to look at each element at least once."*

17.4 Time Management in a 45-Minute Interview

Phase	Suggested Time
Clarifying questions	2-3 minutes
Explaining brute force + optimal approach	5-7 minutes
Coding	15-20 minutes
Testing / dry run	5 minutes
Follow-up questions / variations	Remaining time

⚠️ **Common Mistake** Spending too long "perfecting" the brute force explanation and running out of time to code the optimal solution. Keep your explanation concise, then move to code.

PART IV — YOUR ROADMAP TO MASTERY

Chapter 18: 100 Curated Practice Questions (Roadmap)

This chapter gives you a **high-level roadmap** for how to structure your practice across Beginner, Intermediate, and Advanced levels. The full, detailed 100-question list — with checkboxes to track your progress — is provided as a **separate companion file**: `100 Array Questions Tracker.md`. Use that file daily; use this chapter to understand the overall strategy.

18.1 The 3-Phase Roadmap

Phase	Number of Questions	Goal	Time Suggested
🟢 Beginner	30	Build pattern recognition & basic syntax comfort	2 weeks
🟡 Intermediate	40	Combine patterns, handle tricky edge cases	3 weeks
🔴 Advanced	30	Handle multi-pattern, FAANG-level questions confidently	3 weeks

18.2 How to Practice Effectively (Not Just "Solve and Move On")

Most students practice wrong. They solve a question, see it worked, and move to the next one. This does NOT build interview-readiness. Follow this process instead:

1. **Attempt for 15-20 minutes without looking at the solution.**

2. **If stuck, identify which pattern you think applies** (using the Decision Tree) before peeking at any hints.
3. **After solving (or seeing the solution), write down in one sentence**: "I should have recognized this as [pattern] because of [trigger words]."
4. **Revisit the question after 3 days, then after 7 days** (spaced repetition) — solve it again from scratch, without looking.
5. **Track your confidence level** (Shaky / Comfortable / Mastered) in the tracker file.

> 💡 **Remember** Solving 100 questions passively is worth less than solving 40 questions actively (with the reflection process above) and truly internalizing the patterns behind them.

18.3 Pattern Distribution Across the 100 Questions

Pattern	Approx. Number of Questions
Sliding Window	11
Two Pointer	10
Prefix Sum	6
Prefix Sum + HashMap	5
2D Prefix Sum	3
Difference Array	3
Kadane's Algorithm	4
Frequency Counting	5
Boyer-Moore Voting	2
Cyclic Sort	5
Rotate Array	3
Next Permutation	3
Mixed / Multi-Pattern / Foundational	40

(See `100 Array Questions Tracker.md` for the full list with LeetCode numbers, difficulty, pattern, subpattern, "why solve it," and interview frequency for every single question.)

Chapter 19: Revision Cheatsheet

> 📌 A condensed, one-page-style version of this cheatsheet is also provided separately in `Array_Cheatsheet.md` — print that file for quick pre-interview revision. Below is the full reference version.

19.1 Pattern Recognition Cheatsheet

If the problem says…	Think…
Contiguous / consecutive / subarray / substring / window of size K	Sliding Window

If the problem says…	Think…
Sorted array + pair / palindrome / container / merge	Two Pointer
Repeated range-sum queries	Prefix Sum
Subarray sum = K + negative numbers allowed	Prefix Sum + HashMap
Matrix + repeated sub-rectangle sum queries	2D Prefix Sum
Multiple range updates, read final array	Difference Array
Maximum sum of contiguous subarray	Kadane's Algorithm
Count / most-frequent / anagram / duplicates	Frequency Counting (HashMap)
Appears more than n/2 or n/3 times	Boyer-Moore Voting
Numbers guaranteed in range 1 to N	Cyclic Sort
Rotate by K positions	Reversal Algorithm
Next lexicographical arrangement	Next Permutation

19.2 Time & Space Complexity Cheatsheet

Pattern	Time	Space	Common Problems
Sliding Window	O(n)	O(1) or O(k)	Longest Substring w/o Repeating Chars, Min Size Subarray Sum
Two Pointer	O(n) or $O(n^2)$ for triplets	O(1)	Two Sum II, 3Sum, Container With Most Water
Prefix Sum	O(n) build, O(1) query	O(n)	Range Sum Query, Find Pivot Index
Prefix Sum + HashMap	O(n)	O(n)	Subarray Sum Equals K
2D Prefix Sum	O(rows×cols) build, O(1) query	O(rows×cols)	Range Sum Query 2D
Difference Array	O(1) per update, O(n) rebuild	O(n)	Range Addition, Car Pooling
Kadane's Algorithm	O(n)	O(1)	Maximum Subarray, Max Product Subarray
Frequency Counting	O(n)	O(k)	Valid Anagram, Top K Frequent Elements
Boyer-Moore Voting	O(n)	O(1)	Majority Element, Majority Element II
Cyclic Sort	O(n)	O(1)	Missing Number, Find the Duplicate Number
Rotate Array (Reversal)	O(n)	O(1)	Rotate Array
Next	O(n)	O(1)	Next Permutation

Pattern	Time	Space	Common Problems
Permutation			

19.3 The Decision Tree (Condensed)

```
Continuous subarray/substring?          → Sliding Window
Pair in sorted array?                   → Two Pointer
Repeated range sum queries?             → Prefix Sum
   + negative numbers, sum = K?         → Prefix Sum + HashMap
2D grid range sum queries?              → 2D Prefix Sum
Multiple range updates?                 → Difference Array
Maximum sum contiguous subarray?        → Kadane's Algorithm
Count / frequency?                      → HashMap
Majority element (n/2, n/3)?            → Boyer-Moore Voting
Numbers guaranteed in range 1..N?       → Cyclic Sort
Rotate by K?                            → Reversal Algorithm
Next arrangement?                       → Next Permutation
```

Chapter 20: Final Interview Checklist

Use this checklist before you walk into ANY array-based interview. If you can honestly check every box, you are ready.

✅	Checklist Item
☐	I can read a problem and identify the likely pattern within 2 minutes using the Decision Tree
☐	I can explain the brute force approach out loud, including its time/space complexity
☐	I can explain WHY the optimal approach works, not just HOW to code it (intuition, not memorization)
☐	I can code the optimal solution for all 12 patterns without looking at notes
☐	I know the common mistakes for each pattern and actively avoid them
☐	I always clarify constraints (array size, sorted or not, negative numbers, duplicates) before coding
☐	I test my code with at least one example after writing it
☐	I state time and space complexity after finishing my solution, without being asked
☐	I can recognize when TWO patterns are combined in a single problem
☐	I stay calm and communicate my thinking even when I get stuck

🎯 **Final Interview Insight** The difference between a candidate who gets an offer and one who doesn't is rarely "who knew more algorithms." It's usually "who communicated their thinking clearly, handled edge cases, and stayed calm under pressure." Master the patterns in this handbook — but master your **communication** just as much.

🎉 Congratulations

You've completed **The Complete Array Interview Handbook**. If you've genuinely worked through every chapter, practiced the questions, and internalized the Decision Tree, you are now equipped to confidently tackle array questions at Amazon, Microsoft, Google, Atlassian, Uber, Adobe, and beyond.

> 💡 **Remember** Patterns don't just help you solve array problems — they are reused throughout Strings, Linked Lists, Trees, Graphs, and Dynamic Programming. What you've learned here is a foundation for your **entire** interview preparation journey, not just this one topic.

Next steps: 1. Open `Array_Cheatsheet.md` and revise it the night before any interview. 2. Open `100 Array Questions Tracker.md` and start your Beginner phase today. 3. Revisit this handbook's Decision Tree (Chapter 2) every time you get stuck on a new problem.

Good luck. Go get that offer.